NAND Gate:

$\omega_1 = -1$ $\omega_2 = -1$ bias $= 1.5$

Formula:

$$n = \sum_{i=1}^{n} (x_i \cdot w_i) + b$$

1. Input (0,0)

$n = (0 \cdot -1 + 0 \cdot -1) + 1.5$

$= 1.5 \geq 0$

2. Input (0,1)

$n = (0 \cdot -1 + 1 \cdot -1) + 1.5$

$= 0.5 \geq 0$

2. (1,0)

$n = (1 \cdot -1 + 0 \cdot -1) + 1.5$

$= 0.5 \geq 0$

3. (1,1)

$n = (1 \cdot -1 + 1 \cdot -1) + 1.5$

$= -0.5 \leq 0$

Table Representation:

Input (x_1, x_2)	w_1	w_2	bias	Target (y)	$z = w_1 x_1 + w_2 x_2 + b$	Output
(0,0)	-1	-1	1.5	1	$z = (0 \cdot -1 + 0 \cdot -1) + 1.5$ $= 1.5 \geq 0$	1
(0,1)	-1	-1	1.5	1	$z = (0 \cdot -1 + 1 \cdot -1) + 1.5$ $= 0.5 \geq 0$	1
(1,0)	-1	-1	1.5	1	$z = (0 \cdot -1 + 0 \cdot -1) + 1.5$ $= 0.5 \geq 0$	1
(1,1)	-1	-1	1.5	0	$z = (1 \cdot -1 + 1 \cdot -1) + 1.5$ $= -0.5 \leq 0$	0

CODE:

```
import numpy as np
def perceptron (x1, x2):
    x = np.array([x1, x2])
    weights = np.array([-1, -1])
    bias = 1.5
```

```
output = 1 if z >= 0 else 0
return z, output

inputs = [(0,0), (0,1), (1,0), (1,1)]

print (f"{'Input':{calculated : {'output'}}")

print ("_" * 40)

for x1, x2 in inputs:
    z, output = perceptron (x1, x2)
    input_str = f" ({x1}, {x2})"
    calc_str = f"({x1} * -1) + ({x2} * -1) + 1.5 = {z:.1f}"

print (f"{input_str:<10} | {calc_str:<15} | {output}")
```

Output:

NAND Gate Implementation

Input : [0,0] → calculated Output : 1

Input : [0,1] → Calculated output : 1

Input : [1,0] → Calculated output : 1

Input : [1,1] → Calculated output : 0

RESULT:

Thus, the Logic gate AND, OR and NAND Gate using Single Layer perceptron implemented successfully